

DOCUMENT DE RÉFÉRENCE — CLAUDE CODE

Architecture technique détaillée

Le blueprint d'ingénierie de Gboroly : monorepo TypeScript full-stack, modular monolith NestJS, moteur de compétition indépendant, multi-tenant et mobile-first, pensé pour l'Afrique et le multisport.

Modular Monolith

Competition Engine

Multi-tenant SaaS

Africa-ready

Sommaire

1. Mission du projet
2. Principes architecturaux
3. Stack technique officielle
4. Architecture générale
5. Monorepo
6. Responsabilité des apps
7. Packages
8. Architecture backend
9. Modules backend
10. Multi-tenancy
11. RBAC
12. Modèle de données
13. User
14. Organization
15. Tournament
16. Tournament Category
17. Team
18. Player
19. Registration
20. Competition Engine
21. Format de compétition
22. Round
23. Group
24. Match
25. Match Events
26. Score
27. Classement
28. Tiebreakers
29. Qualifications
30. Bracket
31. Calendrier
32. Match Day
33. Temps réel
34. Notifications
35. WhatsApp
36. Paiements
37. Transaction financière
38. Stockage des fichiers
39. Pages publiques
40. Dashboard organisateur
41. Dashboard principal
42. Création d'un tournoi
43. Import Excel / CSV
44. Export
45. Audit log
46. Soft delete
47. API REST
48. Validation API
49. Gestion des erreurs
50. Cache
51. Jobs asynchrones
52. Idempotence
53. Concurrency et scores
54. Transactions
55. Performance
56. Indexation database
57. Sécurité
58. Environnements
59. Variables d'environnement
60. Logging
61. Monitoring
62. Tests
63. Tests E2E critiques
64. Tests de non-régression
65. CI/CD
66. Design system
67. Responsive
68. PWA
69. Offline / connectivité
70. SEO
71. Architecture multisports
72. Abstraction sport
73. Marketplace future
74. Abonnements futurs
75. Analytics
76. Event-driven interne
77. Outbox pattern
78. API publique future
79. QR codes
80. Admin Gboroly
81. Pas de fichier monstre
82. Pas de duplication
83. Frontend = présentation
84. Database = source of truth
85. Automatisation avec contrôle
86. Workflow de développement
87. Critère de fin de phase
88. Documentation obligatoire
89. Mode de travail obligatoire
90. Interdictions
91. Architecture cible
92. Vision technique finale
93. Objectif du MVP
94. Instruction finale

1 Mission du projet

Construire Gboroly, une plateforme SaaS permettant de digitaliser et automatiser l'intégralité du processus d'organisation d'un tournoi sportif.

Le sport initial est le **Maracana**, avec un lancement prioritaire en Côte d'Ivoire, puis une expansion vers l'Afrique francophone et, à terme, plusieurs sports et pays.

REEMPLACER

Papier → Excel → WhatsApp → appels → calculs manuels → tableaux imprimés

PAR UNE PLATEFORME UNIQUE PERMETTANT

Création → Inscriptions → Équipes → Joueurs → Paiements → Format → Calendrier → Terrains → Matches → Résultats → Classements → Qualifications → Phases finales → Communication → Public → Statistiques → Historique

2 Principes architecturaux non négociables

2.1 Ne pas commencer par les fonctionnalités secondaires

Le cœur du produit est :

- | | |
|----------------------------|-------------------|
| 1. Organisation du tournoi | 6. Matches |
| 2. Gestion des équipes | 7. Résultats |
| 3. Gestion des joueurs | 8. Classements |
| 4. Moteur de compétition | 9. Qualifications |
| 5. Calendrier | 10. Page publique |

Les paiements, WhatsApp, marketplace, statistiques avancées, etc. viennent ensuite.

2.2 Le moteur de compétition doit être indépendant de l'interface

Le moteur de compétition ne doit jamais dépendre de React, Next.js, NestJS Controllers ou composants UI. Il doit pouvoir être exécuté indépendamment.

```
Tournament
  ↓
Competition Engine
  ↓
Standings
  ↓
Qualification
  ↓
Next Round
```

RÈGLE

Le moteur doit être testable avec de simples données JSON.

3 Stack technique officielle

Utiliser une architecture **TypeScript full-stack**.

Couche	Technologies
Frontend	Next.js, React, TypeScript, Tailwind CSS, composants UI réutilisables, React Hook Form, Zod, TanStack Query, PWA.
Backend	Node.js, NestJS, TypeScript, REST API, WebSocket / Socket.IO pour le temps réel.
Base de données	PostgreSQL, Prisma ORM.
Cache / Jobs	Redis, BullMQ.
Stockage	Compatible S3 pour : logos, photos, documents, affiches, bannières, fichiers exportés.
Authentification	Backend : email + mot de passe, téléphone, OTP, access token, refresh token, gestion des sessions, récupération de mot de passe, vérification email/téléphone. Hash : Argon2id .
Infrastructure	Docker pour le développement local. Architecture facilement déployable sur un cloud moderne.

4 Architecture générale

CHOIX FONDATEUR

Ne **PAS** commencer par une architecture microservices. Utiliser dans un premier temps un **MODULAR MONOLITH** avec séparation stricte des domaines.



5 Monorepo

Utiliser un monorepo avec **pnpm + Turborepo**.

```
gboroly/
├── apps/
│   ├── web/
│   ├── api/
│   └── worker/
├── packages/
│   ├── ui/
│   ├── config/
│   ├── types/
│   ├── validation/
│   ├── competition-engine/
│   ├── database/
│   └── utils/
├── docs/
├── docker/
├── scripts/
├── .env.example
├── docker-compose.yml
├── package.json
├── pnpm-workspace.yaml
├── turbo.json
└── README.md
```

6 Responsabilité des applications

App	Responsabilités
<code>apps/web</code> (Next.js)	Site marketing, authentification UI, dashboard organisateur, dashboard administrateur, interface match day, pages publiques, pages équipes, pages joueurs, pages tournoi, marketplace future.
<code>apps/api</code> (NestJS)	Authentification, autorisation, logique métier, API REST, WebSockets, gestion des tournois, gestion des équipes, moteur d'orchestration, paiements, notifications, audit, administration.
<code>apps/worker</code> (BullMQ + Redis)	Génération de calendrier, génération de PDF, génération d'affiches, notifications, emails, tâches WhatsApp, statistiques lourdes, imports, exports, traitements différés.

7 Packages

`packages/competition-engine`

Contient toute la logique du moteur de compétition. Ce package ne doit dépendre d'aucun framework frontend.

```

createTournament()
createRound()
createGroup()
generateFixtures()
calculateStandings()
applyTieBreakers()
determineQualifiedTeams()
generateBracket()
advanceTeams()
handleForfeit()
recalculateTournament()

```

packages/types

Types TypeScript partagés :

Tournament

Team

Player

Match

Standing

Round

Group

Payment

Registration

packages/validation

Schémas Zod partagés :

CreateTournamentSchema

CreateTeamSchema

CreatePlayerSchema

CreateMatchSchema

CreatePaymentSchema

packages/ui — Design system Gboroly

Button

Input

Select

Modal

Drawer

Table

Card

Badge

Tabs

Toast

Dialog

DatePicker

DataTable

Pagination

EmptyState

LoadingState

ConfirmDialog

8 Architecture backend

Utiliser une architecture en couches :

Presentation ↓ **Application** ↓ **Domain** ↓ **Infrastructure**

Couche	Contenu
Presentation	Controllers, DTO, Guards, Pipes, WebSocket gateways.
Application	Use Cases, services applicatifs, Commands, Queries. Ex. : CreateTournament, RegisterTeam, ApproveRegistration, GenerateTournamentSchedule, StartMatch, RecordMatchEvent, FinishMatch, RecalculateStandings, AdvanceQualifiedTeams.
Domain	Entities, Value Objects, Domain rules, Domain events, Competition rules.
Infrastructure	Prisma, Redis, S3, Email, SMS, WhatsApp, Payment providers, External APIs.

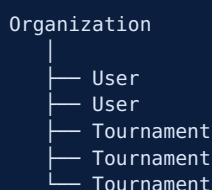
9 Modules backend

Créer les modules suivants :



10 Multi-tenancy

Gboroly est un SaaS multi-tenant. Une organisation peut posséder plusieurs tournois.



Toutes les données privées doivent être rattachées à une organisation lorsque pertinent. Le champ `organization_id` doit être présent dans les principales tables métier.

11 RBAC

Créer un système de permissions :



SÉCURITÉ

Ne jamais vérifier uniquement le rôle côté frontend. Toutes les permissions doivent être vérifiées côté backend.

12 Modèle de données

Entités principales :

User	Round	Payment
Organization	Group	PaymentTransaction
OrganizationMember	GroupTeam	PaymentMethod
Role	Match	
Permission	MatchTeam	Sponsor
	MatchEvent	
Sport	MatchOfficial	Announcement
Tournament		Notification
TournamentCategory	Standing	
		Complaint
Team	Venue	
TeamMember	Field	Media

Player	Referee	AuditLog
Registration	TournamentFormat FormatTemplate	Subscription

13 User

```

User
- id
- firstName
- lastName
- email
- phone
- passwordHash
- avatarUrl
- status
- emailVerifiedAt
- phoneVerifiedAt
- createdAt
- updatedAt

```

14 Organization

```

Organization
- id
- name
- slug
- logoUrl
- description
- country
- city
- timezone
- currency
- status
- createdAt
- updatedAt

```

15 Tournament

```

Tournament
- id
- organizationId
- sportId
- name
- slug
- description
- logoUrl
- bannerUrl
- country
- city
- venueDescription
- registrationStart
- registrationEnd
- status
- visibility
- timezone
- currency
- maxTeams
- registrationFee
- createdAt
- updatedAt

```

```
- startDate
- endDate
```

Statuts

DRAFT

READY

PUBLISHED

ONGOING

COMPLETED

ARCHIVED

CANCELLED

16 Tournament Category

Un tournoi peut contenir plusieurs catégories.

Tournoi Abidjan 2027

```
├─ Senior
├─ U20
└─ Féminin
```

Chaque catégorie peut avoir : son format, ses équipes, ses règles, son calendrier, ses matches, son classement.

17 Team

```
Team
- id
- organizationId
- name
- shortName
- slug
- logoUrl
- captainId
- managerId
- status
```

18 Player

```
Player
- id
- firstName
- lastName
- dateOfBirth
- photoUrl
- phone
- position
- nationality
- status
```

HISTORIQUE DU JOUEUR

Le joueur ne doit pas être directement enfermé dans un seul tournoi. Créer une relation `TeamPlayer` permettant à un joueur d'avoir un historique.

19 Registration

```
Registration
- id
- tournamentId
- categoryId
- teamId
- status
- submittedAt
- approvedAt
- rejectedAt
- rejectionReason
- paymentStatus
```

20 Competition Engine

CŒUR TECHNIQUE DE GBOROLY

Créer une abstraction `CompetitionFormat` avec des implémentations concrètes.

RoundRobin

GroupStage

SingleElimination

DoubleElimination

GroupToPlayoffs

CustomCompetition

21 Format de compétition

```
Tournament
  ↓
Category
  ↓
Competition
  ↓
Round
  ↓
Group
  ↓
Matches
```

22 Round

```
Round 1 = Group Stage
Round 2 = Quarter Finals
Round 3 = Semi Finals
Round 4 = Final
```

Chaque round possède : `roundType` , `order` , `status` , `qualificationRules` .

23 Group

```
Groupe A
Team A
```

Team B
Team C
Team D

Chaque équipe possède une position dans le groupe.

24 Match

```
Match
- id
- tournamentId
- categoryId
- roundId
- groupId
- homeTeamId
- awayTeamId
- venueId
- fieldId
- refereeId
- scheduledAt
- status
- homeScore
- awayScore
- winnerId
- createdAt
- updatedAt
```

Statuts

SCHEDULED READY LIVE HALFTIME FINISHED POSTPONED CANCELLED FORFEIT

25 Match Events

Créer une table `MatchEvent`. Types :

GOAL OWN_GOAL YELLOW_CARD RED_CARD SUBSTITUTION INJURY PENALTY FOUL
OTHER

Chaque événement contient : `matchId`, `teamId`, `playerId`, `eventType`, `minute`, `additionalTime`, `metadata`, `createdAt`.

26 Score

LE SCORE EST UNE DONNÉE MÉTIER

Ne jamais considérer le score comme une simple information affichée.

Lorsqu'un score est modifié :

```
Score Update
↓
Validate
↓
Persist
↓
Update Match
↓
Update Standing
↓
Update Statistics
↓
Check Qualification
↓
```

```
Update Next Round
  ↓
Emit Real-Time Event
  ↓
Update Public Page
```

27 Classement

Créer un service `StandingService`. Il doit pouvoir calculer :

played wins draws losses goalsFor goalsAgainst goalDifference points
fairPlay position

28 Tiebreakers

Les critères doivent être configurables.

1. Points
2. Goal Difference
3. Goals For
4. Head To Head
5. Fair Play
6. Random Draw

RÈGLE

Ne pas coder ces règles directement dans l'interface.

29 Qualifications

Le système doit savoir automatiquement classer 1^{er}/2^e de chaque groupe et les injecter dans les phases finales.

```
1er Groupe A      Quarter Final 1
2e  Groupe A      →  Quarter Final 2
1er Groupe B      ...
2e  Groupe B

QF1 = A1 vs B2
QF2 = B1 vs A2
```

Le moteur doit supporter des règles de qualification configurables.

30 Bracket

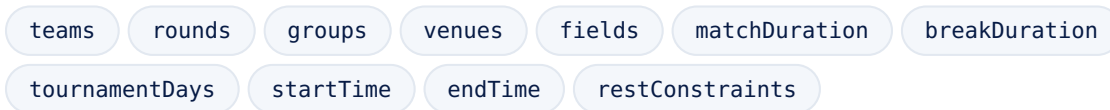
Le bracket doit être généré à partir des rounds. Ne pas créer un système complètement séparé du moteur de compétition.

Competition Engine ↓ **Rounds** ↓ **Bracket**

Le bracket est une représentation graphique des données du moteur.

31 Calendrier

Créer un `ScheduleGenerator`. Inputs :



Le générateur doit essayer d'éviter

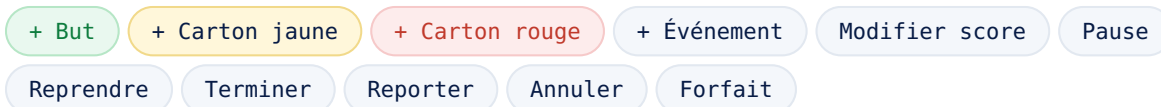
deux matchs simultanés pour une même équipe, repos insuffisant, conflit de terrain, conflit d'arbitre, surcharge d'un terrain.

L'organisateur doit pouvoir modifier manuellement le résultat généré.

32 Match Day

Créer une interface dédiée `/matches/[matchId]/control`, extrêmement rapide.

Actions principales



33 Temps réel

Utiliser WebSockets / Socket.IO. Événements :



Un utilisateur consultant une page publique ne doit pas avoir besoin de rafraîchir la page pour voir un changement de score.

34 Notifications

Créer un système d'abstraction `NotificationProvider`. Providers futurs :



Ainsi le cœur métier ne dépend jamais directement de WhatsApp ou d'un fournisseur particulier.

35 WhatsApp

Prévoir une couche `WhatsAppService`. Exemples :

```
Votre match est prévu à 15h00.  
Terrain : Terrain 2  
Gboroly  
  
—  
  
Résultat
```

Le système doit pouvoir changer de fournisseur WhatsApp sans modifier le domaine métier.

36 Paiements

Créer une abstraction `PaymentProvider` avec : `createPayment()`, `verifyPayment()`, `refundPayment()`, `getPaymentStatus()`.

Prévoir les providers : `Wave` `Orange Money` `MTN Money` `Moov Money` `Card` `Manual Payment`

L'intégration réelle de chaque fournisseur pourra être ajoutée progressivement.

37 Transaction financière

Chaque paiement doit permettre de distinguer : `grossAmount`, `paymentProcessingFee`, `platformFee`, `organizerAmount`, `platformAmount`, `currency`.

Poste	Montant
Inscription (brut)	50 000 FCFA
Frais plateforme	2 500 FCFA
Frais paiement	1 000 FCFA
Organisateur	46 500 FCFA
Gboroly	2 500 FCFA
Provider	1 000 FCFA

Les règles exactes de commission seront configurables.

38 Stockage des fichiers

RÈGLE

Ne pas stocker les fichiers directement dans PostgreSQL.

Stocker uniquement en base : `url`, `key`, `mimeType`, `size`, `metadata`. Les fichiers vont dans le stockage S3.

39 Pages publiques

```
/t/[tournamentSlug]
/t/[tournamentSlug]/matches
/t/[tournamentSlug]/results
/t/[tournamentSlug]/standings
/t/[tournamentSlug]/teams
/t/[tournamentSlug]/players
```

```
/t/[tournamentSlug]/bracket
/t/[tournamentSlug]/statistics
/t/[tournamentSlug]/information
```

Ces pages doivent être : rapides, responsive, indexables, partageables, optimisées mobile.

40 Dashboard organisateur

```
/dashboard
/dashboard/tournaments
/dashboard/tournaments/[id]
/dashboard/tournaments/[id]/overview
/dashboard/tournaments/[id]/teams
/dashboard/tournaments/[id]/players
/dashboard/tournaments/[id]/matches
/dashboard/tournaments/[id]/schedule
/dashboard/tournaments/[id]/standings
/dashboard/tournaments/[id]/bracket
/dashboard/tournaments/[id]/payments
/dashboard/tournaments/[id]/referees
/dashboard/tournaments/[id]/sponsors
/dashboard/tournaments/[id]/communications
/dashboard/tournaments/[id]/settings
```

41 Dashboard principal

Afficher : Mes tournois, Tournois en cours, Équipes inscrites, Matches aujourd'hui, Matches terminés, Paiements, Revenus, Alertes, Activité récente.

42 Création d'un tournoi

Utiliser un wizard.

Étape	Contenu	Étape	Contenu
Step 1	Informations générales	Step 6	Inscriptions
Step 2	Sport / catégorie	Step 7	Paiements
Step 3	Format	Step 8	Vérification
Step 4	Règles	Step 9	Publication
Step 5	Terrains		

SAVE DRAFT

À chaque étape. L'utilisateur ne doit jamais perdre ses données.

43 Import Excel / CSV

Prévoir :

Upload ↓ **Parse** ↓ **Validate** ↓ **Preview** ↓ **Show Errors** ↓ **Confirm** ↓ **Import**

Ne jamais importer directement sans validation.

44 Export

Formats :

Exports : Teams, Players, Matches, Schedule, Standings, Statistics, Payments, Tournament Summary.
Les gros exports doivent être traités par le worker.

45 Audit log

Chaque action sensible doit être enregistrée.

```
AuditLog
- userId      - entityId
- organizationId - oldValue
- action      - newValue
- entityType  - ipAddress
              - userAgent
              - createdAt
```

Actions sensibles

46 Soft delete

Pour les données critiques : ne pas supprimer physiquement directement. Utiliser `deletedAt` lorsque pertinent.

47 API REST

Versionner l'API : `/api/v1/`

```
POST /api/v1/auth/login
POST /api/v1/auth/register

GET  /api/v1/tournaments
POST /api/v1/tournaments
GET  /api/v1/tournaments/:id
PATCH /api/v1/tournaments/:id

GET  /api/v1/tournaments/:id/teams
```

```
POST /api/v1/tournaments/:id/teams
GET /api/v1/tournaments/:id/matches
POST /api/v1/tournaments/:id/matches

POST /api/v1/matches/:id/events
PATCH /api/v1/matches/:id/score
POST /api/v1/matches/:id/start
POST /api/v1/matches/:id/finish

GET /api/v1/tournaments/:id/standings
GET /api/v1/tournaments/:id/bracket
```

48 Validation API

Toutes les entrées utilisateur doivent être validées.

Frontend ↓ Zod ↓ API ↓ DTO Validation ↓ Domain Validation ↓ Database

RÈGLE

Ne jamais faire confiance au frontend.

49 Gestion des erreurs

Utiliser des erreurs métier explicites, avec des codes stables.

TEAM_ALREADY_REGISTERED

PLAYER_ALREADY_REGISTERED

TOURNAMENT_NOT_PUBLISHED

MATCH_ALREADY_FINISHED

INVALID_SCORE

INVALID_QUALIFICATION

PAYMENT_ALREADY_COMPLETED

FIELD_UNAVAILABLE

INSUFFICIENT_PERMISSION

50 Cache

Redis peut être utilisé pour : sessions, rate limiting, données fréquemment consultées, pages publiques très consultées, résultats temps réel, jobs.

RÈGLE

Ne pas mettre toute la base dans Redis. PostgreSQL reste la source de vérité.

51 Jobs asynchrones

BullMQ. Queues :

email

sms

whatsapp

notifications

exports

imports

pdf

analytics

media

payments

Chaque job doit être : **idempotent**, **retryable**, **observable**.

52 Idempotence

Important pour les paiements et opérations critiques. Utiliser une `Idempotency-Key`.

RÈGLE

Une même requête de paiement envoyée deux fois ne doit jamais créer deux paiements.

53 Concurrency et scores

Le système doit empêcher deux opérateurs de modifier simultanément un résultat de façon incohérente. Utiliser : transactions PostgreSQL, optimistic locking lorsque nécessaire, vérification du statut du match, audit log.

54 Transactions

Les opérations critiques doivent être transactionnelles.

```
Finish Match

BEGIN
  Update Match
  Create Events
  Update Result
  Update Standings
  Update Qualification
COMMIT

-- Si une étape échoue :
ROLLBACK
```

55 Performance

Objectifs : dashboard rapide, pages publiques rapides, faible consommation mobile, pagination systématique, lazy loading, images optimisées, requêtes SQL optimisées, index PostgreSQL adaptés.

RÈGLE

Ne jamais charger des milliers de joueurs/matches en une seule requête.

56 Indexation database

Prévoir notamment des index sur :

organizationId tournamentId categoryId teamId playerId matchId scheduledAt
status createdAt

Les indexes doivent être déterminés à partir des requêtes réelles.

57 Sécurité

- HTTPS
- Secure Cookies
- CSRF protection selon architecture
- XSS protection
- Rate Limiting
- CORS
- Input Validation
- SQL Injection protection via ORM
- File Upload Validation
- RBAC
- Audit Logs
- Password Hashing
- Session Management

RÈGLE

Les secrets ne doivent jamais être commités.

58 Environnements

Créer : development staging production

Fichiers : `.env.example` , `.env.local` . Les secrets réels doivent rester hors Git.

59 Variables d'environnement

DATABASE_URL	S3_SECRET_KEY
REDIS_URL	
	EMAIL_PROVIDER
JWT_SECRET	EMAIL_API_KEY
JWT_REFRESH_SECRET	
	WHATSAPP_API_KEY
S3_ENDPOINT	
S3_BUCKET	PAYMENT_PROVIDER_KEYS
S3_ACCESS_KEY	
	NEXT_PUBLIC_API_URL

60 Logging

Utiliser des logs structurés.

```
{
  "level": "info",
  "event": "match.finished",
  "matchId": "...",
  "tournamentId": "...",
  "userId": "..."
}
```

NE JAMAIS LOGGER

mots de passe, tokens, secrets, données bancaires sensibles.

61 Monitoring

Prévoir : Application errors, API latency, Database errors, Queue failures, Payment failures, WebSocket errors, Authentication failures.

Prévoir une solution de monitoring compatible avec l'infrastructure choisie.

62 Tests

PRIORITÉ

Le moteur de compétition doit avoir une couverture de tests très élevée.

Tests obligatoires : Round Robin Group Stage Elimination Qualification Standings
Tie Breakers Forfeit Draw Penalty Shootout Match Reschedule Team Withdrawal

63 Tests E2E critiques

```
Create Organization → Create Tournament → Create Category
↓
Add Teams → Add Players → Configure Format
↓
Generate Groups → Generate Schedule
↓
Start Match → Add Goal → Finish Match
↓
Calculate Standings → Qualify Teams
↓
Generate Next Round → Publish Result
```

Ce scénario doit fonctionner de bout en bout.

64 Tests de non-régression

Après chaque fonctionnalité importante :

```
pnpm lint
pnpm typecheck
pnpm test
pnpm test:e2e
pnpm build
```

Aucune fonctionnalité existante ne doit être cassée.

65 CI/CD

Git Push ↓ **Lint** ↓ **Typecheck** ↓ **Unit Tests** ↓ **Integration Tests** ↓ **E2E** ↓ **Build** ↓ **Deploy Staging**

La production ne doit être déployée qu'après validation.

66 Design system

Respecter l'identité Gboroly.

Navy #071B45	Blue #1269D3	Light Blue #2D8CFF
Orange #FF6A00	Yellow #FFB20D	Green #18A957
Text #17233A	Gray #64748B	Background #F4F7FB

Style : moderne, sportif, professionnel, énergique, premium, simple, mobile-first.

67 Responsive

Mobile ↓ **Tablet** ↓ **Desktop**

Les opérations critiques doivent être parfaitement utilisables sur téléphone : ajouter un score, ajouter un but, consulter le calendrier, consulter le classement, valider une équipe, consulter un match, déplacer un match.

68 PWA

Prévoir une Progressive Web App. Objectifs : installation sur smartphone, accès rapide, cache des ressources, expérience mobile, possibilité d'améliorer progressivement l'expérience hors connexion.

NOTE MVP

La synchronisation offline complète des données critiques n'est pas obligatoire pour le MVP.

69 Offline / connectivité faible

Le produit doit être conçu pour les réalités des réseaux africains. Prévoir : Optimized payloads, Image compression, Caching, Retry, Graceful degradation, Connection status.

L'interface Match Day doit être particulièrement légère.

70 SEO

Les pages publiques doivent être SEO-friendly. Prévoir :

Metadata

OpenGraph

Twitter/X cards

Canonical URLs

Sitemap

Robots.txt

Structured Data

71 Architecture future multisports

VISION

Ne pas coder Gboroly comme `MaracanaApp` mais comme **Sports Competition Platform**.

Sport ↓ **Competition Rules** ↓ **Tournament** ↓ **Matches**

À terme : `Maracana` `Football` `Basketball` `Volleyball` `Handball` `etc.`

72 Abstraction sport

Créer `SportRules`. Exemple :

```
calculateScore()
calculateStandings()
validateMatch()
calculateStatistics()
handleOvertime()
handlePenaltyShootout()
```

Chaque sport pourra implémenter ses propres règles.

73 Marketplace future

Ne pas développer maintenant, mais préparer les modèles : `TournamentDiscovery`

`TournamentRegistration` `TeamParticipation` `PlatformCommission`

Team discovers tournament → **Register** → **Pay** → **Organizer approves** → **Team participates**

74 Abonnements futurs

Prévoir une entité `Subscription`. Plans futurs : `Free` `Starter` `Pro` `Enterprise`

Ne pas imposer immédiatement le système d'abonnement au MVP si le modèle économique n'est pas encore définitivement arrêté.

75 Analytics

Éviter de construire immédiatement un système analytique complexe. Commencer par les événements métier :

`tournament.created` `team.registered` `payment.completed` `match.started` `match.finished`
`result.published`

Ces événements pourront ensuite alimenter les analytics.

76 Event-driven internal architecture

Prévoir des événements métier. Exemple : `MatchFinished` peut déclencher :

```
MatchFinished
  ↓
UpdateStandings
UpdateStatistics
CheckQualification
PublishResult
NotifyTeams
UpdatePublicPage
```

Cela permet de découpler progressivement les fonctionnalités.

77 Outbox pattern

Pour les événements importants, prévoir progressivement un Outbox Pattern.

```
PostgreSQL Transaction ↓ Match updated ↓ Outbox Event created ↓ Worker ↓ Notifications /
realtime / analytics
```

Cela évite de perdre des événements.

78 API publique future

À terme, certaines données publiques pourront être exposées via API.

```
GET /public/tournaments
GET /public/tournaments/:slug
GET /public/tournaments/:slug/matches
GET /public/tournaments/:slug/standings
```

Cela pourra permettre : sites partenaires, applications tierces, médias sportifs, widgets, écrans de stade.

79 QR codes

Chaque tournoi peut avoir un `QR → Page publique tournoi`. Chaque match peut éventuellement avoir un `QR → Page match`.

80 Admin Gboroly

Créer un espace `/admin`. Fonctionnalités :

Organizations Users Tournaments Sports Payments Transactions Subscriptions
Commissions Reports Complaints Audit Logs System Settings

81 Règle importante — pas de fichier monstre

Ne pas créer un énorme fichier contenant toute la logique. Éviter un `tournament.service.ts` de plusieurs milliers de lignes.

Préférer une organisation `application/` · `domain/` · `infrastructure/` et des services spécialisés.

82 Règle importante — pas de duplication

Une règle métier ne doit exister qu'à un seul endroit.

EXEMPLE

La logique de classement doit vivre dans `competition-engine` — et non dans le frontend, un backend controller ou un database trigger simultanément.

83 Règle importante — frontend = présentation

LE FRONTEND NE DOIT PAS DÉCIDER

qui est qualifié · combien de points · qui gagne · quel match vient ensuite.

Le backend / competition engine décide. Le frontend affiche.

84 Règle importante — database = source of truth

PostgreSQL est la source de vérité. Redis ne doit jamais être considéré comme la source définitive des données.

85 Règle importante — automatisation avec contrôle

Gboroly doit automatiser : Classement, Calendrier, Qualifications, Bracket, Notifications, Statistiques. Mais permettre à l'organisateur de reprendre le contrôle lorsque nécessaire.

Calendrier automatique ↓ **Organisateur vérifie** ↓ **Modification manuelle possible** ↓ **Validation**

86 Workflow de développement Claude Code

Claude Code doit travailler par phases.

Phase 0 — Analyse

Ne pas coder. Produire `ARCHITECTURE.md`, `DATABASE.md`, `DOMAIN-MODEL.md`, `API.md`. Puis vérifier les incohérences.

Phase 1 — Fondations

Monorepo, Next.js, NestJS, PostgreSQL, Prisma, Redis, Docker, CI, Design System.

Phase 2 — Authentication

Register, Login, Logout, Refresh Token, Forgot/Reset Password, Email & Phone Verification, RBAC.

Phase 3 — Organisations

Organization, Members, Roles, Permissions, Invitations.

Phase 4 — Tournois

Create/Edit Tournament, Tournament Wizard, Categories, Visibility, Publication.

Phase 5 — Équipes / Joueurs

Teams, Players, Registrations, Approvals, Team & Player Management.

Phase 6 — Competition Engine ★

Phase la plus importante. Formats, Groups, Rounds, Fixtures, Standings, Tie Breakers, Qualification, Bracket. Ne pas passer à la suite tant que les tests ne sont pas solides.

Phase 7 — Matches

Match, Match Day, Match Events, Score, Officials, Forfeit, Postponement, Cancellation.

Phase 8 — Calendar

Schedule Generator, Fields, Venues, Constraints, Manual Editing.

Phase 9 — Public

Tournament Public Page, Matches, Results, Standings, Teams, Players, Bracket, Statistics.

Phase 10 — Realtime

WebSockets, Live Score, Match Events, Live Standings.

Phase 11 — Communication

Notifications, Email, WhatsApp abstraction, Announcements.

Phase 12 — Payments

Payment abstraction, Manual & Online payment, Transactions, Commissions.

Phase 13 — Advanced features

Sponsors, Advanced Statistics, Exports, Imports, QR Codes, Tournament Duplication, Analytics.

Phase 14 — Marketplace

Tournament Discovery, Team Registration, Online Payment, Platform Commission, Team & Player Profiles, Tournament History.

87 Critère de fin de chaque phase

Claude Code ne doit jamais considérer une phase terminée simplement parce que le code compile. Une phase est terminée uniquement lorsque :

Feature implemented + Unit tests + Integration tests + E2E where relevant + Typecheck + Lint + Build + Documentation + No regression

88 Documentation obligatoire

Créer :



89 Mode de travail obligatoire

Avant chaque nouvelle fonctionnalité :

1. Lire le code existant.
2. Lire la documentation correspondante.
3. Identifier les dépendances.
4. Proposer les fichiers qui seront créés/modifiés.
5. Implémenter.
6. Tester.
7. Corriger.
8. Mettre à jour la documentation.
9. Vérifier la non-régression.

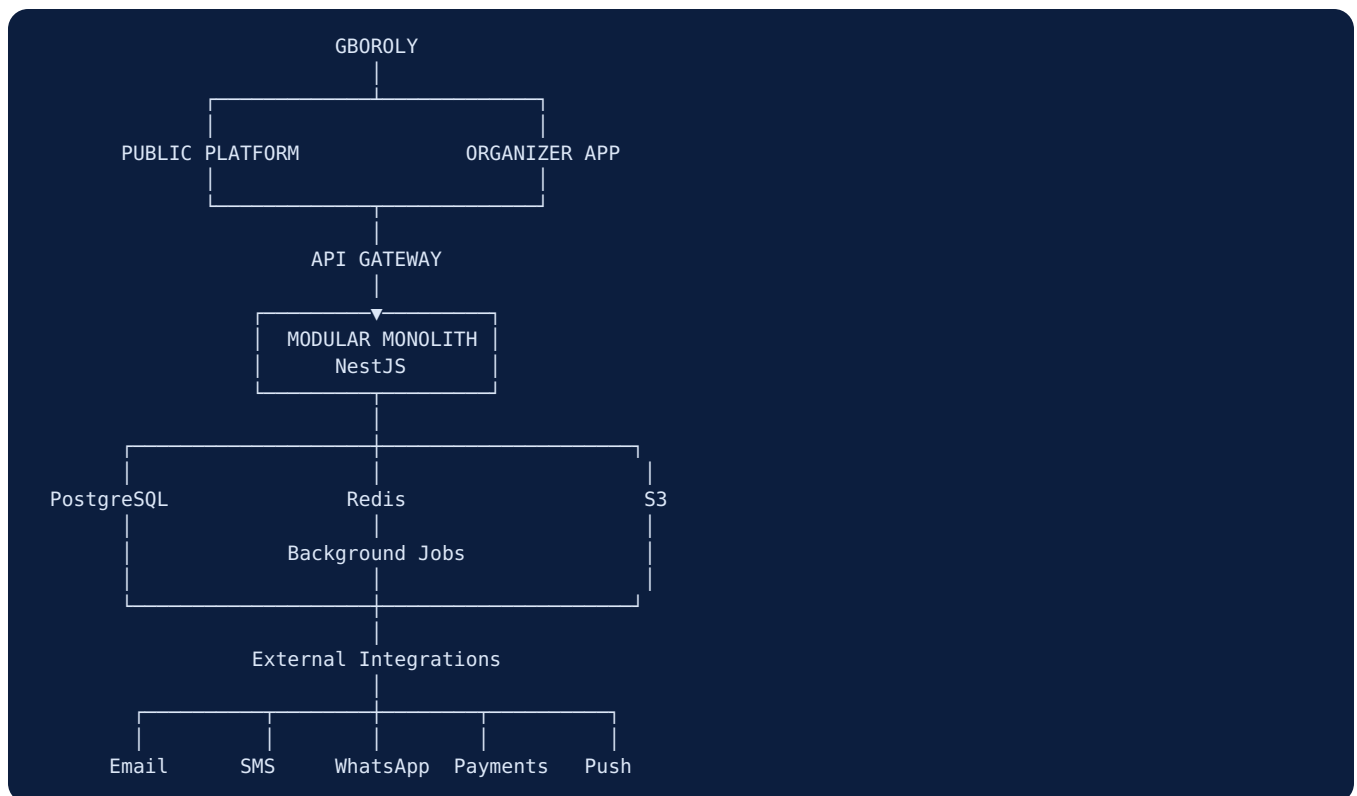
Ne jamais réécrire inutilement une partie fonctionnelle du projet.

90 Interdictions

- ✗ construire toute l'application en une seule étape ;
- ✗ créer des microservices prématurément ;
- ✗ mettre la logique métier dans les composants React ;
- ✗ dupliquer les règles métier ;
- ✗ stocker les fichiers dans PostgreSQL ;
- ✗ mettre les secrets dans Git ;
- ✗ utiliser Redis comme source de vérité ;

- ✗ supprimer définitivement les données critiques sans justification ;
- ✗ contourner le RBAC ;
- ✗ créer des règles sportives directement dans l'UI ;
- ✗ faire confiance aux données envoyées par le frontend ;
- ✗ ignorer les tests.

91 Architecture cible



92 Vision technique finale

Gboroly doit être conçu comme un **moteur de gestion de compétitions sportives** avec une interface SaaS au-dessus, et non comme une simple application de résultats sportifs.

Le système doit donc être :



LE VÉRITABLE PRODUIT EST

THE AFRICAN SPORTS TOURNAMENT OPERATING SYSTEM. Le Maracana est le premier cas d'utilisation.

93 Objectif du MVP

À la fin du MVP, un organisateur doit pouvoir faire ceci **sans utiliser Excel ou papier** :

1. Créer son organisation
2. Créer son tournoi
3. Choisir son format
4. Ajouter ses équipes
5. Ajouter les joueurs
6. Générer les groupes
7. Générer le calendrier
8. Affecter terrains et arbitres
9. Jouer les matchs
10. Saisir les scores
11. Calculer automatiquement les classements
12. Déterminer les qualifiés
13. Générer les phases finales
14. Publier les résultats
15. Permettre au public de suivre le tournoi

CŒUR DE PRODUIT

Si ce workflow fonctionne parfaitement, Gboroly possède son cœur de produit. Tout le reste peut être ajouté progressivement autour de ce noyau.

94 Instruction finale à Claude Code

Construis Gboroly en respectant strictement cette architecture. Ne commence pas directement par coder toutes les fonctionnalités. Commence par analyser ce document et produire :

- | | |
|---|--------------------------------|
| 1. Architecture finale du repository | 8. Stratégie multi-tenancy |
| 2. Diagramme des modules | 9. Architecture temps réel |
| 3. Schéma complet de la base PostgreSQL | 10. Architecture des jobs |
| 4. Diagramme des relations | 11. Architecture des paiements |
| 5. Architecture du Competition Engine | 12. Plan de tests |
| 6. Liste des API endpoints | 13. Roadmap d'implémentation |
| 7. Système RBAC | |

Ensuite, attends la validation de l'architecture avant d'entreprendre les phases de développement.

PRIORITÉ ABSOLUE

Construire un Competition Engine robuste, générique, testable et indépendant de l'interface. Ne pas sacrifier la qualité de l'architecture pour aller plus vite sur les fonctionnalités visuelles.

Gboroly doit être suffisamment bien architecturé pour commencer avec le Maracana en Côte d'Ivoire et évoluer ensuite vers une véritable plateforme africaine de gestion de compétitions sportives.

GBOROLY • THE AFRICAN SPORTS TOURNAMENT OS

Fin du document d'architecture technique — v1.0